

Bridge GeCode Validator

Instructivo de compilacion, uso y referencia tecnica

Validador de sintaxis (dry-run) para el paradigma Bridge Pascal/JSON a GeCode Space. Permite verificar la estructura de un JSON antes de enviarlo al motor de busqueda, sin compilar GeCode ni ejecutar la busqueda.

Archivo	Descripcion
bridge_gecode.y	Gramatica Bison (parser)
bridge_gecode.l	Lexer Flex (scanner)
Makefile	Automatiza compilacion
test_bridge.txt	Ejemplo de entrada para testing

1. Procedimiento de compilacion

1.1 Requisitos previos

Herramienta	Version minima	Funcion
GNU Bison	>= 3.0	Genera el parser (C) a partir de la gramatica .y
GNU Flex	>= 2.6	Genera el lexer (C) a partir de las reglas .l
GCC	>= 9.0	Compila los archivos C generados en un ejecutable

Instalacion en Ubuntu/Debian:

```
sudo apt-get install bison flex gcc
```

1.2 Pasos de compilacion

Paso 1: Bison procesa la gramatica (.y)

```
bison -d -v bridge_gecode.y
```

Flag	Efecto
-d	Genera bridge_gecode.tab.h (tokens para el lexer)
-v	Genera bridge_gecode.output (informe de estados)
(auto)	Genera bridge_gecode.tab.c (el parser en C)

Paso 2: Flex procesa el lexer (.l)

```
flex bridge_gecode.l
```

Genera lex.yy.c. Incluye bridge_gecode.tab.h para conocer los tokens.

Paso 3: GCC compila todo junto

```
gcc -Wall -std=c11 -o bridge bridge_gecode.tab.c lex.yy.c -lm
```

El flag -lm enlaza la libreria matematica (pow, round).

1.3 Compilacion rapida

```
make          # compila todo
make test     # compila y ejecuta con test_bridge.txt
make clean    # limpia archivos generados
```

1.4 Diagrama del proceso

Entrada	Herramienta	Salida
bridge_gecode.y	Bison	bridge_gecode.tab.c + .h

bridge_gecode.l	Flex	lex.yy.c
.tab.c + lex.yy.c	GCC	bridge (ejecutable)
test_bridge.txt	./bridge	Validacion + AST

2. Uso del validador

2.1 Ejecucion

```
./bridge mi_problema.txt
```

El validador parsea las tres secciones y produce:

- Lista de variables con tipo, dominio y valor
- AST (arbol) de cada expresion
- Lista de funciones con sus argumentos
- Errores de sintaxis o referencias invalidas
- Resumen final: conteos y tabla global de sets

2.2 Formato de entrada

```
precision: 1;

variables: {
  horas : integer : [1, 40] : 20;
  costo : integer : [100, 5000] : [150, 160];
  aprobado : logic : [true, false] : true;
  temperatura : float : [15, 45] : 40.0;
  equipo : set : {"Roles": ["dev","qa"]} : "dev";
}

expresiones: {
  horas * costo <= 5000;
  temperatura >= 20.0;
  aprobado = 1 IMPLICA horas >= 10;
}

funciones: {
  distinct : [horas, prioridad] : nil;
  minimize : [costo] : nil;
}
```

2.3 Semantica del campo value

value	Significado	GeCode genera
20 (escalar)	Fijo, sin incertidumbre	rel(*this, var, IRT_EQ, 20)
[150, 160]	Con incertidumbre	IntVar(*this, IntSet({150,160}))
true	Logico fijo	rel(*this, var, IRT_EQ, 1)
[true, false]	Logico con incertidumbre	BoolVar(*this, 0, 1) libre
"dev"	Set fijo: GLB=LUB={dev}	SetVar(*this, {id}, {id})

["a","b"]	Set con incert: GLB={a,b}	SetVar(*this, {a,b}, LUB_completo)
nil	Sin valor asignado	Variable libre en su dominio

2.4 Salida esperada

```
Expr #1:
  [<=]
    [*]
      IDENT(horas)
      IDENT(costo)
      ENTERO(5000)
```

3. El archivo .y (Bison - Parser)

El archivo `bridge_gecode.y` define la **gramatica formal** del lenguaje de entrada. Bison lo procesa y genera un parser LALR(1) en C.

3.1 Estructura del archivo

Seccion	Contenido
%{ ... %}	Prologo C: includes, tipos (Nodo, VarReg), prototipos, tablas globales
%union	Union semantica: tipos posibles de los simbolos (entero, string, nodo AST)
%token	Tokens: simbolos terminales del lexer (NUMERO, AND, IMPLICA, etc.)
%type	Tipos de no-terminales: asocia reglas con tipos de la union
%% ... %%	Reglas gramaticales con acciones semanticas entre { }
Funciones C	Constructores de AST, impresion, validacion, main()

3.2 La gramatica: estructura del Bridge

La regla raiz refleja las tres secciones del JSON:

```
programa
  : decl_precision sec_vars sec_expr sec_funcs
  ;
```

El orden es obligatorio y respeta la invariante I3: variables antes que expresiones, expresiones antes que funciones.

3.3 Precedencia de operadores

Nivel	Operador	Regla	Asoc.
1 (menor)	IMPLICA	<code>expr_impl</code>	Derecha
2	OR	<code>expr_or</code>	Izquierda
3	AND	<code>expr_and</code>	Izquierda
4	NOT	<code>expr_not</code>	Derecha
5	<code>= <> < > <= >= in</code>	<code>expr_comp</code>	No asoci.
6	<code>+ -</code>	<code>expr_arit</code>	Izquierda
7	<code>* /</code>	<code>expr_term</code>	Izquierda
8 (mayor)	<code>func()</code> <code>ident num ()</code>	<code>expr_factor/atom</code>	--

3.4 Acciones semanticas

Cada regla tiene código C que se ejecuta al ser reconocida. En el validador hacen tres cosas: construir nodos AST, registrar en tablas, e imprimir diagnóstico.

```
expr_arit '+' expr_term  
    { $$ = nodo_binario(NODO_SUMA, $1, $3); }
```

\$\$ es el resultado, \$1 el operando izquierdo, \$3 el derecho. \$2 sería el token '+' en sí.

4. El archivo .l (Flex - Lexer)

El archivo `bridge_gecode.l` define el **scanner/lexer**: convierte texto crudo en tokens para el parser.

4.1 Estructura del archivo

Seccion	Contenido
%{ ... %}	Prologo C: includes y referencia a factor_global
%option	Opciones: yylineno (contar lineas), noyywrap (un solo archivo)
%% ... %%	Reglas: cada linea es patron regex -> accion C (retorna token)

4.2 Categorias de tokens

Categoria	Patron	Token	Ejemplo
Palabras clave	"precision" etc.	PRECISION	precision
Tipos	"integer" etc.	TIPO_INTEGER	float
Booleanos	"true", "false"	BOOL_TRUE/FALSE	true
Op. logicos	"AND", "OR", "NOT", "IMPLICACION"	AND, OR...	AND
Comparacion	"=", "<>", "<=" etc.	EQ, NEQ, LEQ...	<=
Func. estandar	"abs"/["]("	FN_ABS etc.	abs(
Float	[0-9]+\.[0-9]+	NUMERO (convertido)	45.3 -> 453
Entero	[0-9]+	NUMERO	100
Identificador	[a-zA-Z_][...]*	IDENTIFICADOR	horas
String	"..."	STRING_LIT	"dev"

4.3 Conversion de floats a la grilla

El mecanismo clave: al encontrar un decimal, lo multiplica por el factor global y redondea:

```
[0-9]+\.[0-9]+ {  
    double val = atof(yytext);  
    yylval.entero = (int)round(val * factor_global);  
    return NUMERO;  
}
```

Precision	Factor	Entrada	Token
1	10	45.3	453
1	10	1.25	13 (redondeo)

2	100	3.14	314
---	-----	------	-----

Consecuencia clave: Bison nunca ve floats. Todo llega como entero. La grilla queda garantizada por construcción.

4.4 Distinción función vs. identificador

El lexer usa lookahead para distinguir funciones estándar:

```
"abs"/[ \t]*"("  -> FN_ABS      (abs seguido de parentesis)
[a-zA-Z_][...]*  -> IDENTIFICADOR (abs sin parentesis)
```

Las funciones no estándar (distinct, count) siempre son IDENTIFICADOR; el parser las diferencia por su posición en la sección 3.

5. Relacion Lexer - Parser

Etapas	Componente	Entrada	Salida
1	Flex (.l)	Texto crudo caracter a caracter	Secuencia de tokens
2	Bison (.y)	Secuencia de tokens	AST + tablas + validacion

Bison llama a `yy lex ( )` cada vez que necesita el siguiente token. Flex lee caracteres hasta reconocer un patron y retorna el token. El valor semantico viaja a traves de `yy lval`.

Ejemplo: como se procesa `temperatura >= 20.0` con precision 1:

Flex lee	Retorna	yy lval	Bison reconoce
temperatura	IDENTIFICADOR	.str="temperatura"	--
>=	GEQ	--	--
20.0	NUMERO	.entero=200	expr_comp: IDENT GEQ ENTERO